Retriever-final-class

29 July 2026 14:34

RAG Pipeline Intro.

Documents
↓
Parsing / Loading
↓
Chunking
↓
Embeddings
↓
Vector Store
↓
Retriever
↓
LLM
↓
Final Answer

What is retriever

A retriever takes a user query and returns the most relevant documents or chunks from a knowledge source.

A retriever normally does not generate the final answer itself; it collects the relevant context required to generate the ans wer.

A retriever is a component that:

Takes a user query
↓
Searches the knowledge source
↓
Returns the most relevant documents

A retriever's job is to find relevant information based on the user's question.

User question
↓
Retriever searches the stored documents
↓
Returns the most relevant chunks
↓
The LLM uses those chunks to generate the answer

Simple example

Suppose a vector database contains 1,000 chunks extracted from PDF documents.
The user asks:

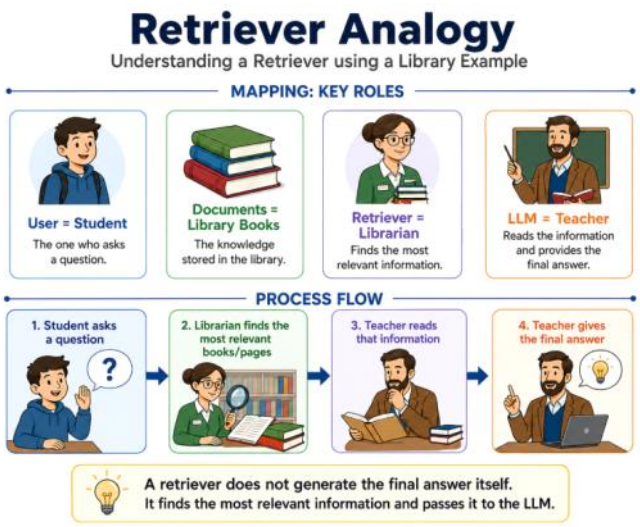
What is semantic chunking?
The retriever does not send all 1,000 chunks to the LLM. It searches the database and returns only the relevant chunks:

Chunk 12 → Definition of semantic chunking
Chunk 48 → Example of semantic chunking
Chunk 91 → Advantages of semantic chunking
These relevant chunks are then passed to the LLM.

Simple analogy

Think of a retriever as a librarian:

User = Student
Documents = Library books
Retriever = Librarian
LLM = Teacher
The student asks a question. The librarian finds the most relevant books or pages and gives them to the teacher. The teacher reads those pages and generates the final answer.



code reminder from langchain

```
retriever = vector_store.as_retriever(
   search_type="mmr",
   search_kwargs={
      "k": 4,        # Final number of documents to return
      "fetch_k": 20   # Candidate documents considered by MMR
   }
)

# Retrieve relevant documents
documents = retriever.invoke(
   "What is a vector database?"
)

# Display the retrieved content and metadata
for document in documents:
   print(document.page_content)
   print(document.metadata)
   print("-" * 50)
```

k = final number of results
filter = metadata restriction
score_threshold = minimum acceptable relevance score
fetch_k = number of candidates fetched before MMR
lambda_mult = balance between relevance and diversity in MMR

search_type → decides which search algorithm runs
similarity → returns the most similar chunks
similarity_score_threshold → returns only the chunks that pass the threshold
mmr → returns relevant and diverse chunks
search_kwargs → configures the selected search algorithm

Similarity Search Methods

1. Cosine Similarity

Measures the **angle/direction similarity** between two vectors.

$$\text{Cosine Similarity}(A, B) = \frac{A \cdot B}{\| A \| \| B \|}$$

Higher score = More similar
Most commonly used for text embeddings.

2. Euclidean Distance — L2

Measures the **straight-line distance** between two vectors.

$$d(A, B) = \sqrt{\sum_{i=1}^{n} (A_i - B_i)^2}$$

Smaller distance = More similar

3. Dot Product / Inner Product

Multiplies corresponding vector values and adds them.

$$A \cdot B = \sum_{i=1}^{n} A_i B_i$$

Higher score = More similar

With normalized vectors, dot product becomes equivalent to cosine similarity.

Easy Summary

Method	Best result
Cosine similarity	Highest score
Euclidean distance	Lowest distance
Dot product	Highest score

These are the three primary similarity metrics commonly supported by vector databases such as Pinecone.

Metadata Filtering

A retriever should not rely only on semantic similarity. It should also support structured constraints using document metadat a.

Metadata helps the retriever limit the search to documents that satisfy specific conditions such as department, year, documen t type, source, user role, or tenant.

Example Metadata

```
metadata = {
   "department": "HR",
   "year": 2026,
   "document_type": "policy",
   "access_role": "manager"
}
```

User Query

Show the HR leave policy for 2026.

Metadata Filter

```
filter = {
   "department": "HR",
   "year": 2026
}
```

The retriever will search only the documents whose metadata matches:

department = HR
year = 2026

It will ignore documents from other departments or years, even when their content is semantically similar to the query.

Common Types of Metadata Filters

- **Exact-match filters**
  Match an exact metadata value.

  ```
  {"department": "HR"}
  ```
- **Range filters**
  Match values within a range.

  ```
  {"year": {"$gte": 2024, "$lte": 2026}}
  ```
- **Boolean filters**
  Combine multiple conditions using AND, OR, or NOT.

  ```
  {
     "$and": [
        {"department": "HR"},
        {"year": 2026}
     ]
  ```

}

- **Date filters**
  Retrieve documents created or updated within a specific date range.
- **Department filters**
  Restrict retrieval to departments such as HR, Finance, Legal, or Engineering.
- **Document-type filters**
  Restrict retrieval to policies, reports, invoices, manuals, or contracts.
- **Source filters**
  Search only selected PDFs, websites, databases, or repositories.
- **Tenant filters**
  Ensure that users can retrieve documents only from their own organization or tenant.
- **Role-based access filters**
  Restrict documents according to roles such as employee, manager, HR, or administrator.
- **Pre-filtering and post-filtering**
  Decide whether metadata restrictions are applied before or after the retrieval operation.

Vector databases commonly support metadata restrictions during retrieval. These filters reduce the search space and help retu rn only relevant and permitted documents.

The exact filter syntax may differ between vector databases such as Chroma, Pinecone, Qdrant, Weaviate, and Elasticsearch.

Pre-filtering:
Filter first → Search later

Post-filtering:
Search first → Filter later

Pre-filtering

Pre-filtering means applying metadata conditions **before** running vector or keyword search.

All stored documents
↓
Apply metadata filter
↓
Allowed documents only
↓
Similarity or keyword search
↓
Final results

Example

Suppose the vector database contains 10,000 documents:

HR documents = 1,000
Finance documents = 3,000
Engineering documents = 4,000
Legal documents = 2,000
The user asks:

Show the HR leave policy for 2026.
The filter is:

```
filter = {
   "department": "HR",
   "year": 2026
}
```

With pre-filtering:

10,000 documents
↓
Filter department = HR and year = 2026
↓
Only 150 permitted documents remain
↓
Similarity search runs on those 150 documents
↓
Most relevant HR leave-policy chunks are returned

Code Example

```
retriever = vector_store.as_retriever(
   search_type="similarity",
   search_kwargs={
      "k": 4,
```

```
    "filter": {
      "department": "HR",
      "year": 2026
    }
  }
)
documents = retriever.invoke(
  "Show the HR leave policy for 2026."
)
```

Advantages of Pre-filtering

- Searches a smaller document set
- Reduces irrelevant results
- Improves security
- Supports tenant isolation
- Prevents unauthorized documents from entering the candidate list
- Can improve retrieval speed

Pre-filtering is generally preferred for strict authorization because restricted documents are excluded before retrieval begins.

Post-filtering

Post-filtering means running retrieval first and applying metadata conditions afterward.

```
All stored documents
      ↓
Similarity or keyword search
      ↓
Top candidate documents
      ↓
Apply metadata filter
      ↓
Final allowed results
```

Example

Suppose the retriever first returns the top five semantically similar documents:

```
Result 1 → Finance leave policy, 2026
Result 2 → HR leave policy, 2025
Result 3 → Legal leave guideline, 2026
Result 4 → HR leave policy, 2026
Result 5 → Engineering leave policy, 2026
```

Now the filter is applied:

```
filter = {
  "department": "HR",
  "year": 2026
}
```

After post-filtering, only one result remains:

```
Result 4 → HR leave policy, 2026
```

The retriever originally fetched five documents, but four were removed after retrieval.

Post-filtering Example

```
documents = vector_store.similarity_search(
  "Show the HR leave policy for 2026.",
  k=5
)
filtered_documents = [
  document
  for document in documents
  if document.metadata.get("department") == "HR"
  and document.metadata.get("year") == 2026
]
```

Limitations of Post-filtering

- It may return too few final results
- Relevant permitted documents may never enter the initial top-k
- Unauthorized documents may enter the intermediate candidate set
- It is less suitable for strict access control
- A larger initial k may be required

For example:

Initial retrieval returns top 5
↓
4 results fail the filter
↓
Only 1 final result remains
Even though more valid HR documents may exist in the database, they may not have appeared in the original top five.

Retrieval Types

Sparse Retrieval
Sparse retrieval searches using exact keywords and term matching.
Example:

Query: "employee leave policy"
Returns documents containing words such as:
employee, leave, policy
Common methods:

BM25
TF-IDF
Keyword Search

Dense Retrieval
Dense retrieval uses embeddings to understand the semantic meaning of the query.
Example:

Query: "How many days off can employees take?"
It can retrieve:
"Employees are entitled to 20 days of annual leave."
The exact words may be different, but the meaning is similar.

Hybrid Retrieval
Hybrid retrieval combines sparse and dense retrieval.

Keyword/BM25 Search
+
Vector Search
↓
Combined Results
Example:

Query: "HR leave policy 2026"
Sparse retrieval matches exact terms such as:

HR
leave policy
2026
Dense retrieval finds semantically similar content such as:

employee annual vacation guidelines
Hybrid retrieval combines both results for better accuracy.

Query Transformation
Query transformation improves the user's original query before retrieval so the system can find more relevant information. It may rewrite the query, add related terms, or break a complex query into smaller questions.

1. **Query Rewriting**

Query rewriting converts an unclear, incomplete, or conversational query into a clearer standalone search query.
Example:

Original query:
"What did he say about it?"
Rewritten query:
"What did the CEO say about the 2026 acquisition?"
This is especially useful in conversational RAG, where the current question depends on previous messages.

Original query
↓
Clearer standalone query
↓
Retriever

2. Query Expansion
Query expansion adds synonyms, related terms, acronyms, spelling variations, or alternative phrases to the original query.
Example:

Original query:
"employee leave policy"
Expanded query:
"employee leave policy OR vacation policy OR annual leave guidelines"
Another example:

Original term:
"car"
Expanded terms:
"car, automobile, vehicle"
Query expansion broadens the search and helps retrieve documents that use different words for the same concept.

Original query
↓
Add related terms or synonyms
↓
Broader retrieval

3. Query Decomposition
Query decomposition breaks a complex question into smaller and simpler sub-questions.
Example:

Original query:
"Compare the revenue of Company A and Company B in 2025
and explain why their growth rates were different."
It can be decomposed into:

Sub-query 1:
What was Company A's revenue in 2025?
Sub-query 2:
What was Company B's revenue in 2025?
Sub-query 3:
What factors affected Company A's growth?
Sub-query 4:
What factors affected Company B's growth?
The system retrieves information for each sub-query and combines the results to answer the original question. Query decomposition is useful for comparison, multi-hop, and complex questions.

Complex query
↓
Multiple smaller sub-queries
↓
Retrieve evidence for each query
↓
Combine the results

Reranking
Reranking is a second-stage process that takes documents returned by an initial retriever, scores them more accurately against the user query, and reorders them so that the most relevant documents are sent to the LLM.

Benefits of Reranking

Reranking can:

- Improve the ordering of retrieved documents
- Remove weak candidates using a relevance threshold
- Reduce irrelevant context sent to the LLM
- Reduce unnecessary input tokens
- Improve evidence quality
- Work on results from sparse, dense, or hybrid retrieval

Official production implementations such as Elasticsearch and Cohere accept an initial candidate set and reorder it according to query relevance; Elasticsearch also supports candidate-window size, score thresholds, filters, and chunk-level rescoring for long documents.

Limitation

Reranking adds:

- Additional latency
- Additional computation
- Additional inference cost

Therefore, it should normally be applied only to a limited candidate set —not to every document in the database.

Reranking is a **second-stage retrieval process** that takes an initial set of retrieved documents, calculates a more accurate relevance score for each document, and rearranges them from most relevant to least relevant.

User Query
↓
Initial Retriever
(BM25, Vector Search, or Hybrid Search)
↓
Top Candidate Documents
↓
Reranker
↓
Reordered by Relevance
↓
Top Documents Sent to the LLM

Why Is Reranking Required?

The initial retriever must search thousands or millions of documents quickly. Therefore, it normally uses a fast retrieval me thod such as:

- BM25
- Vector similarity search
- Hybrid retrieval

Fast retrieval provides good candidates, but their original order may not be perfectly accurate.
A reranker applies a more powerful model only to this smaller candidate set. This creates a practical balance:

Initial Retrieval → Fast and broad
Reranking → Slower but more accurate
Production search systems use this multi-stage architecture because an expensive ranking model can be applied to a small candidate set rather than the entire document collection.

Simple Example

Suppose the user asks:

How many days of annual leave do employees receive?
The initial retriever returns these candidates:

1. Remote Work Policy
2. Sick Leave Policy
3. Annual Leave Policy
4. Leave Carry-Forward Policy
5. Employee Attendance Policy

These documents are related to employees and leave, but the most useful document is not ranked first.
The reranker evaluates every candidate against the original query:

Annual Leave Policy → 0.95
Leave Carry-Forward Policy → 0.76
Sick Leave Policy → 0.31
Employee Attendance Policy → 0.19
Remote Work Policy → 0.08
The reranker then produces a better order:

1. Annual Leave Policy
2. Leave Carry-Forward Policy
3. Sick Leave Policy
4. Employee Attendance Policy
5. Remote Work Policy

Only the highest-ranked documents are passed to the LLM.

How a Cross-Encoder Reranker Works

A standard embedding retriever usually encodes the query and documents separately:

Query → Query Vector
Document → Document Vector

↓
Similarity Score
Because document vectors can be created and stored in advance, this approach is fast enough to search large collections.
A cross-encoder reranker processes the query and one candidate document **together**:

[Query + Candidate Document]
↓
Cross-Encoder
↓
Relevance Score
This joint processing allows the model to examine detailed relationships between the words in the query and the document. How ever, every query-document pair must be processed separately, making cross-encoders more computationally expensive than first-stage vector retrieval.
For 20 candidate documents, the reranker conceptually evaluates:

Query + Document 1 → Score
Query + Document 2 → Score
Query + Document 3 → Score
...
Query + Document 20 → Score
It then sorts the documents by these scores.

Real Production Flow

A practical production RAG pipeline commonly follows this pattern:

1. User submits a query
↓
2. Apply metadata and security filters
↓
3. Retrieve a broad candidate set
using BM25, vector, or hybrid search
↓
4. Send the candidate set to a reranker
↓
5. Calculate query-document relevance scores
↓
6. Sort candidates by reranker score
↓
7. Apply an optional minimum-score threshold
↓
8. Send the best documents to the LLM
For example:

1,000,000 stored chunks
↓
Retriever selects 50 candidates
↓
Reranker reorders those 50 candidates
↓
Top 5 chunks are passed to the LLM
The values 50 and 5 are only examples. In production, candidate count and final result count are tuned according to accuracy, latency, model limi ts, token budget, and cost. Elasticsearch exposes this candidate window as rank_window_size, and reranking services accept a query plus a candidate document list and return relevance-ranked results.

Was Reranking Introduced for RAG?

No. Reranking existed in **information retrieval and search systems before modern RAG**.
Traditional search systems already used multi-stage or cascade ranking:

Fast candidate generation
↓
More accurate ranking stages
↓
Final search results
The 2011 work *A Cascade Ranking Model for Efficient Ranked Retrieval* formalized a multi-stage ranking architecture designed to balance search effectiveness and computational efficiency.
In 2019, *Passage Re-ranking with BERT* demonstrated that BERT could be adapted to score query-passage pairs and substantially improve passage-ranking performance. This work helped popularize transformer-based semantic reranking, but it did not invent the general reranking concept.
RAG later adopted the same established idea:

Search documents first
↓
Rerank the candidates

↓
Give the best evidence to the LLM

Reranking vs Retrieval

Retrieval	Reranking
Searches the complete collection	Processes only retrieved candidates
Optimized for speed and recall	Optimized for ranking precision
Returns an initial candidate set	Reorders that candidate set
Commonly uses BM25 or embeddings	Commonly uses a cross-encoder or another ranking model
First stage	Second stage

Retriever finds possible documents.
Reranker decides which of those documents are most relevant.

Reranking vs Reciprocal Rank Fusion

They are not the same.

Reciprocal Rank Fusion

RRF combines ranked lists produced by multiple retrievers:

BM25 Results ──┐
├── RRF → Combined Ranking
Vector Results ──┘

RRF mainly uses the **positions of documents in the input rankings**. It does not need to jointly understand every query-document pair.

Semantic Reranking

A semantic reranker evaluates the relationship between the query and each candidate document:

Combined Candidates
↓
Query-Document Model
↓
New Relevance Scores
↓
Final Ranking

Therefore, a complete hybrid production pipeline may use both:

BM25 Search + Vector Search
↓
RRF
↓
Cross-Encoder Reranker
↓
Best Context for LLM

Main Reranking Approaches

1. Pointwise Reranking

Each candidate is scored independently:

Query + Document A → 0.91
Query + Document B → 0.67
Query + Document C → 0.22
The documents are sorted by score.

2. Pairwise Reranking

The model compares two documents and determines which is more relevant:

For this query:
Document A or Document B?

3. Listwise Reranking

The model considers several candidates together and produces an ordered list.

Input:
Document A, B, C, D
Output:
C, A, D, B
The 2019 multi-stage BERT-ranking work describes pointwise monoBERT and pairwise duoBERT models arranged in a multi-stage ranking architecture.

Multimodal Retriever

A multimodal retriever retrieves relevant information across different modalities, such as text and images. It uses multimoda l embedding models to represent compatible modalities in a shared vector space, allowing text -to-image, image-to-text and image-to-image similarity search.

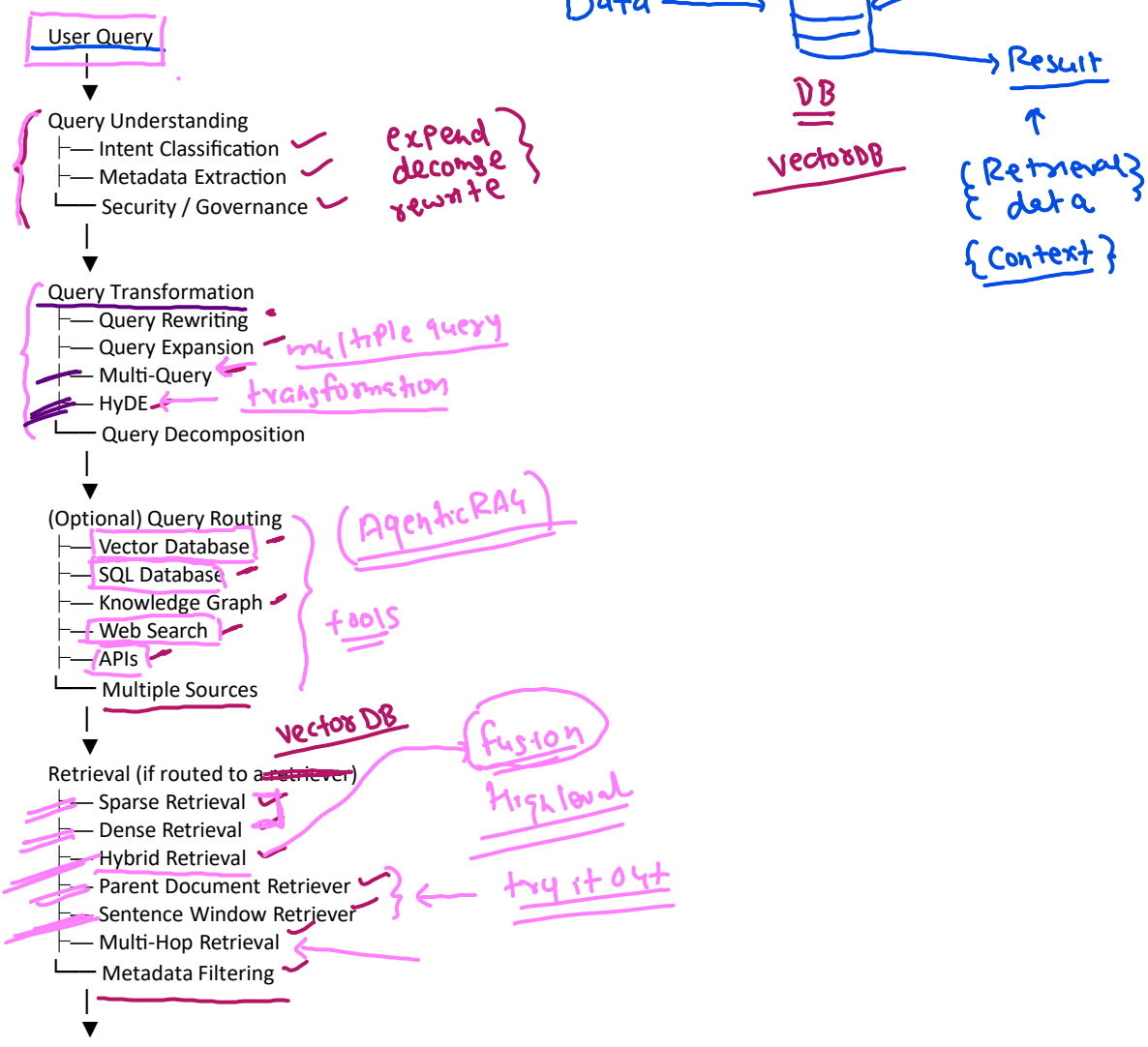
Text-to-Image

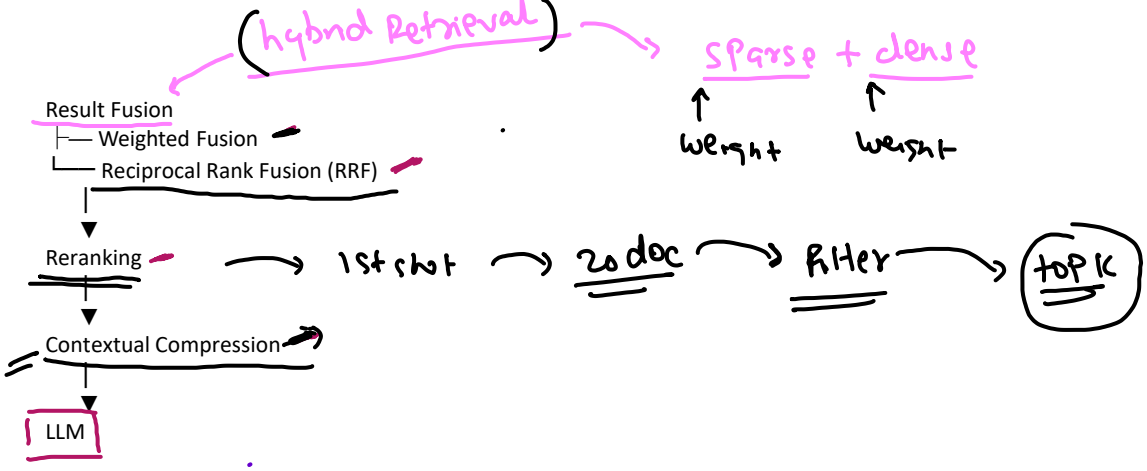
Text Query
↓
Text Encoder
↓
Shared Embedding Space
↓
Search Image Vectors
↓
Relevant Images

Image-to-Image

Image Query
↓
Image Encoder
↓
Image Embedding Space
↓
Search Image Vectors
↓
Similar Images

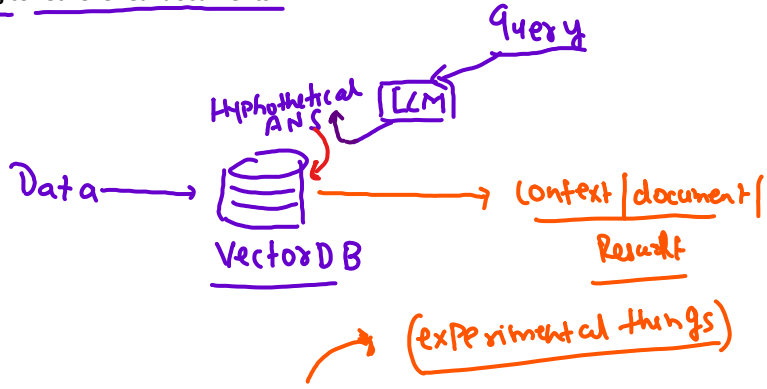
Summarization:





HyDE = Hypothetical Document Embeddings

HyDE = Generate a hypothetical answer first, then use its embedding to retrieve real documents.



Simple flow:
User Query
↓
LLM creates a hypothetical answer/document
↓
That hypothetical text is converted into an embedding
↓
Vector DB searches for real documents similar to it
↓
Relevant real documents are returned

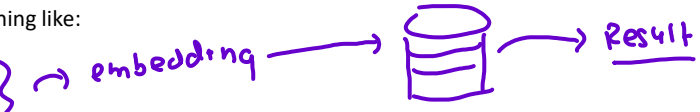
Example

User asks:

How does Llama 2 improve safety?

Instead of directly embedding this short query, HyDE first creates something like:

Llama 2 improves safety using supervised safety fine-tuning, RLHF, red teaming, and safety evaluation.
Then this hypothetical answer is embedded and used for retrieval.



Why?

Because a full hypothetical answer is often **semantically closer to the actual document content** than a short user question.

Multi-Query Retriever

It creates **multiple versions of the same user query**, runs retrieval for each one, then merges the results.

User Query
↓
Generate multiple query variations
↓
Search for each query
↓
Merge + remove duplicates
↓
Final relevant documents

query transformation → expend (query expension)

Example:

Original:
How does Llama 2 improve safety?
Variations:
1. What safety techniques are used in Llama 2?
2. How was Llama 2 safety fine-tuned?
3. How does Llama 2 reduce unsafe responses?

HyDE

HyDE does **not** create multiple questions.
It creates a **hypothetical answer/document**, embeds that, and uses it to search real documents.

Query
↓
Hypothetical answer
↓

Embedding
↓
Vector search

Multi-Query	HyDE
Generates multiple queries	Generates a hypothetical answer
Searches using several query versions	Searches using the answer embedding
Main goal: improve recall	Main goal: improve semantic matching

Evaluation

Parent Document vs Sentence Window

Parent Document Retriever
→ Search small chunk, return larger parent section

Sentence Window Retriever
→ Search one sentence, return nearby sentences

good result

HyDE | multiquery
Query transformation

Parent Document Retrieval

Parent Document Retriever = Search small, return big.
It searches using **small chunks**, but returns the **larger parent document or section** that contains that chunk.

Large Document
↓
Split into small child chunks
↓
Create embeddings for child chunks
↓
User query searches child chunks
↓
Best child chunk is found
↓
Return its larger parent section

Example
Suppose a policy document has a 2,000-token section.
It is split into:

Child Chunk 1 → 300 tokens
Child Chunk 2 → 300 tokens
Child Chunk 3 → 300 tokens
...
User asks:

How many annual leave days are allowed?
Retriever finds:

Child Chunk 3
But instead of returning only that 300-token chunk, it returns the **full parent section**, maybe 1,500–2,000 tokens.
Why use it?
Small chunks are better for **accurate retrieval**, while larger parent sections provide **better context**.

Sentence Window Retriever

Sentence Window Retriever = Search a small sentence, return that sentence with its surrounding context.
It searches using a **single sentence or very small chunk**, but when that sentence matches, it also returns the **nearby sentences around it**.

Document
↓
Split into sentences
↓
Create embeddings for each sentence
↓
User query searches the sentences
↓
Best matching sentence is found
↓
Return that sentence + nearby sentences

Example
Document contains:

Sentence 8 → Llama 2 uses supervised fine-tuning.
Sentence 9 → Human preference data is collected.
Sentence 10 → RLHF is used to further align the model.
Sentence 11 → Reward models score model responses.
Sentence 12 → PPO is used during optimization.
User asks:

How is Llama 2 aligned using human feedback?
The retriever may match:

Sentence 10
But instead of returning only Sentence 10, it returns a window such as:

Sentence 8
Sentence 9
Sentence 10
Sentence 11
Sentence 12
Why use it?
Because one sentence may be very precise for retrieval, but it may not contain enough context for the LLM.

Multi-Hop Retrieval

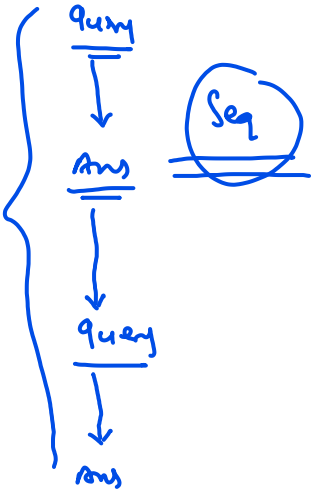
Multi-Hop Retrieval = Retrieve → use the result to retrieve again → combine evidence.
Multi-hop retrieval is used when **one retrieval is not enough** to answer the question.
The system retrieves one piece of information, uses that result to form the next query, retrieves again, and continues until it has enough evidence.

User Query
↓
Retrieve first piece of information
↓
Use that result to create the next query
↓
Retrieve second piece of information
↓
Combine the evidence
↓
Final answer
Example
User asks:

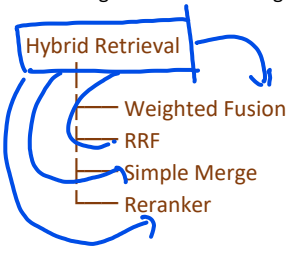
Who founded the company that developed Llama 2?
Hop 1:

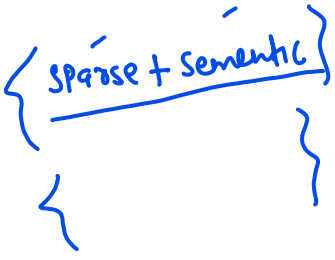
Which company developed Llama 2?
→ Meta
Hop 2:

Who founded Meta?
→ Mark Zuckerberg and co-founders
Then the system combines both hops to answer.
Why use it?
Because some questions require information from **multiple documents or multiple retrieval steps**.



Hybrid Retrieval = Use multiple retrievers
But it **doesn't define how to merge their outputs**.
The merge can be done using:





Case 1: Without Fusion (Simple Merge)
Suppose BM25 returns:

1. Doc A
2. Doc B
3. Doc C

Dense returns:

1. Doc D
2. Doc B
3. Doc E

Simply merge:

Doc A
Doc B
Doc C
Doc D
Doc E

Problem:

- No proper ranking
- BM25 results dominate
- Dense results may appear too low
- Not a good production approach

Case 2: Weighted Fusion

BM25 Score
+
Dense Score
↓
Weighted Formula
↓
Final Ranking
Uses scores.

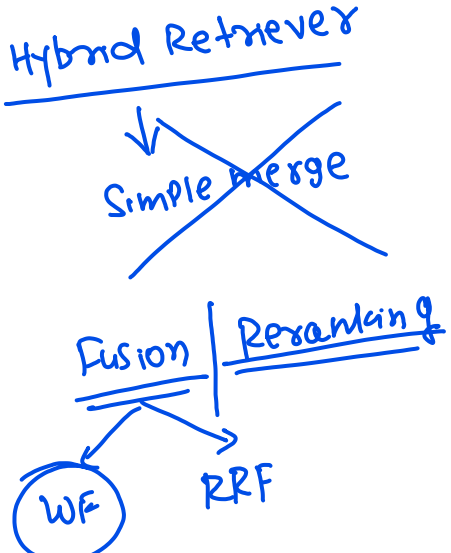
Case 3: RRF

BM25 Rank
+
Dense Rank
↓
RRF Formula
↓
Final Ranking
Uses ranks.

Case 4: Reranking (Very Common)
Some systems don't use Weighted Fusion or RRF.
Instead:

BM25
\
------ Merge Candidates ------
/
Dense /
↓
Top 50 Documents
↓
Cross Encoder Reranker
↓
Final Top 5
Here the reranker decides the final ranking.

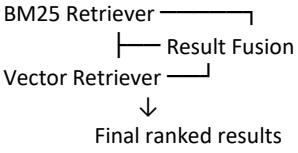
Result Fusion



Result Fusion is used when **multiple retrievers return different ranked lists**, and you want to combine them into one final ranking.

Weighted Fusion	**RRF**
Combines raw/normalized scores	Combines rank positions
Uses explicit weights	Original RRF does not require weights
Score normalization may be needed	Score scales do not need to match

Simple difference
So the easiest way to remember:
Weighted Fusion = score-based fusion
RRF = rank-based fusion



1. Weighted Fusion
Weighted Fusion combines the **scores** from multiple retrievers using weights.
Example:

BM25 score = 0.8
Vector score = 0.9
Weights:
BM25 = 0.4
Vector = 0.6
Final score:

Final Score
=
(0.4 × 0.8)
+
(0.6 × 0.9)
= 0.86
Then documents are sorted using the final combined score.

BM25 Scores + Vector Scores
↓
Apply weights
↓
Combined score
↓
Final ranking
Weighted Fusion = combine scores using weights.
Important: the scores should be made comparable/normalized if the retrievers use different score scales.

2. Reciprocal Rank Fusion (RRF)
RRF combines **rank positions**, not raw scores.
Suppose:

BM25 Ranking
1. Doc A
2. Doc B
3. Doc C
Vector search returns:

Vector Ranking
1. Doc B
2. Doc C
3. Doc A
RRF gives each document points based on where it appears in each ranking:

$$RRF(d) = \sum_i \frac{1}{k + \text{rank}_i(d)}$$

A document that appears near the top in multiple lists gets a higher final RRF score.

BM25 ranking ──┐
├── RRF
Vector ranking ──┘
↓
Combined ranking

RRF = combine rankings using rank positions.

Maximal Marginal Relevance (MMR)

MMR first retrieves the most relevant document, then selects the remaining documents by balancing query relevance and diversi ty, avoiding documents that are too similar to those already selected.

Goal: Retrieve relevant documents while avoiding duplicate or highly repetitive documents.

MMR Formula

$$MMR(D_i) = \lambda \times Sim(D_i, Q) - (1 - \lambda) \times \max_{D_j \in S} Sim(D_i, D_j)$$

Where:

- **Q** = User Query
- **D_i** = Candidate document
- **S** = Documents already selected
- **Sim(D_i,Q)** = Similarity between the candidate document and the user query
- **Sim(D_i,D_j)** = Similarity between the candidate document and the already selected document
- **λ (lambda)** = Controls the balance between relevance and diversity

Step 1: User Query

How does Llama 2 improve safety?
The retriever first fetches **fetch_k = 5** candidate documents.

Document	Query Similarity
Doc A	0.95
Doc B	0.90
Doc C	0.88
Doc D	0.84
Doc E	0.80

Suppose: k = 2
We need only **2 final documents**.

Step 2: Select the First Document
The first document is simply the **most relevant**.

Selected

Doc A
Now: S = {Doc A}

Step 3: Compute MMR for the Remaining Documents
Assume the similarity of each remaining document with **Doc A** is:

Candidate	Query Similarity	Similarity with Doc A
Doc B	0.90	0.95
Doc C	0.88	0.40
Doc D	0.84	0.20
Doc E	0.80	0.10

Let: λ = 0.5

The formula becomes:

$$MMR = 0.5 \times QuerySimilarity - 0.5 \times DocumentSimilarity$$

Doc B

$0.5(0.90) - 0.5(0.95)$

$= 0.45 - 0.475$

$= -0.025$

Doc C

$0.5(0.88) - 0.5(0.40)$

$= 0.44 - 0.20$

$= 0.24$

Doc D

$0.5(0.84) - 0.5(0.20)$

$= 0.42 - 0.10$

$= 0.32$

Doc E

$0.5(0.80) - 0.5(0.10)$

$= 0.40 - 0.05$

$= 0.35$

Final MMR Scores

Document	MMR Score
Doc B	-0.025
Doc C	0.24
Doc D	0.32
Doc E	**0.35**

MMR chooses:

Doc E

Final Result

Normal Similarity Search

1. Doc A
2. Doc B

Both documents may contain almost the same information.

MMR

1. Doc A
2. Doc E

Doc E is slightly less relevant but provides different information, increasing diversity.

Visual Flow

User Query
↓
Retrieve Top 5 Documents
↓
Select Best Document (Doc A)
↓
Compare Remaining Documents
↓
High Query Similarity?
+
Low Similarity with Doc A?
↓
Choose Next Document

Easy Memory Trick
Think of recommending YouTube videos.
Without MMR

Python Tutorial Part 1
Python Tutorial Part 2
Python Tutorial Part 3
Python Tutorial Part 4
Almost the same content.
With MMR

Python Tutorial
Python Project
Python Interview Questions
Python Best Practices
Still relevant to Python, but much more diverse.

Contextual Compression

Contextual Compression means: **First retrieve relevant documents, then remove the parts that are not useful for the current query.**

Simple flow:

User Query
↓
Retriever gets relevant documents
↓
Compressor checks those documents against the query
↓
Irrelevant text is removed
↓
Only useful context is sent to the LLM
Example
Suppose a retrieved HR policy chunk has 1,000 tokens:

- Leave policy
- Dress code
- Work-from-home rules
- Travel reimbursement
- Holiday calendar

User asks:

How many annual leave days do employees get?
Contextual compression may keep only:

Employees are entitled to 20 days of annual leave per year.
Unused leave can be carried forward up to 5 days.
and remove the rest.
Why use it?
It helps reduce:

- irrelevant context
- token usage

- LLM cost
- noise in the prompt

and can improve answer quality.

Important difference from reranking

Reranking
= Reorders documents
Contextual Compression
= Removes irrelevant documents or irrelevant parts inside documents
So the easiest way to remember is:

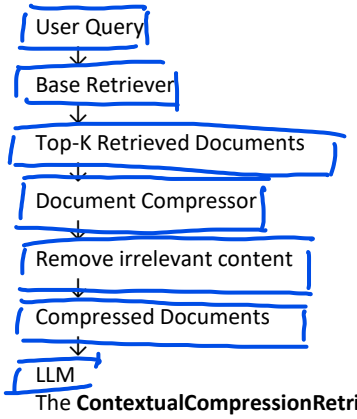
Retriever finds the documents.
Reranker orders them.
Contextual Compression trims them.

How Contextual Compression Works

The main idea is:

Retrieve broadly first, then compress the retrieved documents using the user's query so that only the relevant information is sent to the LLM.

Step-by-Step Flow



The **ContextualCompressionRetriever** is simply a wrapper around an existing retriever.
It works in two stages:

1. **Retrieve** relevant documents using any retriever (Vector Search, BM25, Hybrid, etc.).
2. **Compress** those retrieved documents based on the current query before sending them to the LLM.

Example

Suppose the retriever returns three chunks.

Retrieved Chunk 1

HR Policy
Leave Policy
Employees receive 20 days of annual leave.
Dress Code
Employees must wear formal clothes.
Office Timing
9 AM – 6 PM.

Retrieved Chunk 2

Travel Policy
Flight booking
Hotel booking
Taxi reimbursement

Retrieved Chunk 3

Medical Insurance
Health insurance
Dental insurance
Vision insurance

The user asks:

How many annual leave days are employees entitled to?

Compression Stage
The compressor analyzes the **query** and the **retrieved documents**.

Query
+
Retrieved Documents
↓
Document Compressor
It extracts only the relevant information.
Output:

Employees receive 20 days of annual leave.
Everything else is removed.

What exactly is the compressor doing?
A **Document Compressor** can work in different ways depending on the implementation.
1. LLMChainExtractor
Uses an LLM.

Query
+
Document
↓
LLM
↓
Extract only relevant sentences
Example:
Document:

Leave Policy
20 annual leave days
Dress code
Travel reimbursement
Office timing
Compressed:

20 annual leave days
The LLM extracts only the information relevant to the query instead of returning the whole document.

2. Embeddings Filter
Instead of rewriting the document, it filters documents (or smaller chunks) using embedding similarity.

Retrieved Documents
↓
Embedding Similarity
↓
Keep relevant ones
↓
Discard the rest
No LLM call is required, so this approach is faster and cheaper.

3. Cross-Encoder / Reranker
A reranker can also act as the compressor.

Retrieved Documents
↓
Cross Encoder
↓
Remove low-relevance documents
Only the most relevant documents continue through the pipeline.

Why not simply reduce k?
Suppose

Retriever
↓

Top 3 Documents
What if
Document 1

1000 tokens
Only

20 tokens
are actually useful?
Reducing

k
will not solve this.
You'll still send

1000 tokens
to the LLM.
Instead
Contextual Compression

1000 tokens
↓

20 useful tokens
This is why contextual compression is different from simply retrieving fewer documents.

Production Flow

User Query
↓
Retriever
↓
Top 20 Documents
↓
Contextual Compression
↓
Top 20 become only the relevant passages
↓
LLM
Notice:
The number of documents may remain

20
but each document is much shorter.

Difference from Reranking
People often confuse these.
Reranking

Retriever
↓
Reorder Documents
↓
LLM
Only the order changes.

Contextual Compression

Retriever
↓
Compress Document Content
↓
LLM

The **content inside the documents changes**.

Simple Example
Retriever returns

Document
1000 words
Reranker

1000 words
↓

Still 1000 words
Only the ranking changes.
Contextual Compression

1000 words
↓

120 words
Only the relevant information remains.

Concept	LangChain Class / Function	Import Statement
MMR	vector_store.as_retriever(search_type="mmr")	*(No import required – available from any VectorStore)*
Multi-Query Retriever	MultiQueryRetriever	from langchain_classic.retrievers.multi_query import MultiQueryRetriever
HyDE	HypotheticalDocumentEmbedder	from langchain_classic.chains.hyde.base import HypotheticalDocumentEmbedder
Parent Document Retriever	ParentDocumentRetriever	from langchain_classic.retrievers import ParentDocumentRetriever
Hybrid Retrieval	EnsembleRetriever + BM25Retriever + VectorStore Retriever	from langchain_classic.retrievers import EnsembleRetrieverfrom langchain_community.retrievers import BM25Retriever
Reciprocal Rank Fusion (RRF)	EnsembleRetriever (internally uses weighted_reciprocal_rank())	from langchain_classic.retrievers import EnsembleRetriever
Weighted Fusion (Weighted RRF)	EnsembleRetriever(weights=[...])	from langchain_classic.retrievers import EnsembleRetriever
Contextual Compression	ContextualCompressionRetriever	from langchain_classic.retrievers.contextual_compression import ContextualCompressionRetriever
LLM-based Compression	LLMChainExtractor	from langchain_classic.retrievers.document_compressors import LLMChainExtractor
Embedding-based Filtering	EmbeddingsFilter	from langchain_classic.retrievers.document_compressors import EmbeddingsFilter
Cross-Encoder Reranking	CrossEncoderReranker	from langchain_classic.retrievers.document_compressors import CrossEncoderReranker
Cross-Encoder Model	HuggingFaceCrossEncoder	from langchain_community.cross_encoders import HuggingFaceCrossEncoder
LLM Listwise Reranking	LLMListwiseRerank	from langchain_classic.retrievers.document_compressors import LLMListwiseRerank
Sentence Window Retrieval	No dedicated built-in retriever	Custom implementation (typically metadata + custom retrieval logic)
Multi-Hop Retrieval	No dedicated MultiHopRetriever	Implement using LangGraph / iterative retriever calls
Graph Traversal Retrieval	GraphVectorStoreRetriever	from langchain_community.graph_vectorstores import GraphVectorStoreRetriever

1. MMR

Directly available through as_retriever():

```
retriever = vector_store.as_retriever(
   search_type="mmr",
   search_kwargs={
      "k": 4,
      "fetch_k": 20,
      "lambda_mult": 0.5
   }
)
```

LangChain documents fetch_k and lambda_mult specifically for MMR.

2. Multi-Query Retriever

Class:

```
from langchain_classic.retrievers.multi_query import MultiQueryRetriever
```

Usage:

```
multi_query_retriever = MultiQueryRetriever.from_llm(
   retriever=vector_store.as_retriever(),
   llm=llm,
   include_original=True
)
```

Then:

```
docs = multi_query_retriever.invoke(
   "How does Llama 2 improve safety?"
)
```

It uses an LLM to generate multiple queries, retrieves for each one, and returns the unique union of results.

3. HyDE

LangChain has:

```
from langchain_classic.chains.hyde.base import (
   HypotheticalDocumentEmbedder
)
```

Core class:

```
HypotheticalDocumentEmbedder
```

Example:

```
from langchain_openai import OpenAI, OpenAIEmbeddings
base_embeddings = OpenAIEmbeddings(
   model="text-embedding-3-small"
)
hyde_embeddings = HypotheticalDocumentEmbedder.from_llm(
   llm=OpenAI(),
   base_embeddings=base_embeddings,
   prompt_key="web_search"
)
```

Then use hyde_embeddings as the embedding function for the vector store/query workflow.
The class specifically generates a hypothetical document for the query and embeds it.

Important current-LangChain point

The dedicated HypotheticalDocumentEmbedder exists under langchain_classic, but for new production code I would also teach the **LCEL implementation**:

```
Query
→ Prompt
→ LLM
→ hypothetical text
→ embeddings.embed_query()
→ vector search
```

because it makes the mechanism much clearer.

4. Parent Document Retriever

Direct class:

```
from langchain_classic.retrievers import ParentDocumentRetriever
```

or:

```
from langchain_classic.retrievers.parent_document_retriever import (
   ParentDocumentRetriever
)
```

Typical supporting pieces:

```
from langchain.storage import InMemoryStore
from langchain_text_splitters import RecursiveCharacterTextSplitter
```

Conceptually:

```
parent_splitter = RecursiveCharacterTextSplitter(
   chunk_size=2000
)
```

```
child_splitter = RecursiveCharacterTextSplitter(
   chunk_size=300
)
store = InMemoryStore()
retriever = ParentDocumentRetriever(
   vectorstore=vector_store,
   docstore=store,
   child_splitter=child_splitter,
   parent_splitter=parent_splitter
)
```
Then add documents:

```
retriever.add_documents(documents)
```
Search:

```
docs = retriever.invoke(
   "How was RLHF performed?"
)
```
LangChain's implementation searches embedded child chunks and then returns their larger parent documents.

5. Hybrid Retrieval

For LangChain, the most straightforward implementation is:

```
BM25Retriever
+
VectorStoreRetriever
+
EnsembleRetriever
```
Imports:

```
from langchain_community.retrievers import BM25Retriever
from langchain_classic.retrievers import EnsembleRetriever
```
Example:

```
bm25 = BM25Retriever.from_documents(chunks)
bm25.k = 5
dense = vector_store.as_retriever(
   search_kwargs={"k": 5}
)
hybrid = EnsembleRetriever(
   retrievers=[bm25, dense],
   weights=[0.5, 0.5]
)
```
Then:

```
docs = hybrid.invoke(
   "Llama 2 grouped query attention"
)
```

6. RRF

LangChain's EnsembleRetriever uses **weighted Reciprocal Rank Fusion**.
Class:

```
from langchain_classic.retrievers import EnsembleRetriever

ensemble = EnsembleRetriever(
   retrievers=[
      bm25,
      dense
   ],
   weights=[
      0.5,
      0.5
   ]
)
```
Internally it exposes:

```
weighted_reciprocal_rank(...)
```

So your class can practically show:

BM25 ranking
+
Dense ranking
↓
EnsembleRetriever
↓
Weighted RRF

7. Weighted Fusion

Here one clarification is important.
If by **Weighted Fusion** you mean:

0.4 × normalized BM25 score
+
0.6 × normalized dense score
then EnsembleRetriever is **not exactly that**.
LangChain's EnsembleRetriever(weights=[...]) performs:
Weighted Reciprocal Rank Fusion
rather than generic raw-score weighted addition.
So for teaching:

Weighted RRF
→ EnsembleRetriever(weights=[...])

Generic score-based Weighted Fusion
→ custom code / vector DB specific hybrid implementation
For pure score fusion you would usually manually normalize both score lists and combine them.

8. Contextual Compression

Direct class:

```
from langchain_classic.retrievers import (
   ContextualCompressionRetriever
)
```

Basic structure:

```
compression_retriever = ContextualCompressionRetriever(
   base_retriever=base_retriever,
   base_compressor=compressor
)
```

Then:

```
docs = compression_retriever.invoke(query)
```

It wraps a base retriever and compresses its results.

9. Contextual Compression with LLM Extraction

Use:

```
from langchain_classic.retrievers.document_compressors import (
   LLMChainExtractor
)

compressor = LLMChainExtractor.from_llm(llm)
compression_retriever = ContextualCompressionRetriever(
   base_retriever=dense_retriever,
   base_compressor=compressor
)
```

This actually extracts the query-relevant portions of document content.

10. Contextual Compression with Embedding Filter

Class:

```
from langchain_classic.retrievers.document_compressors import (
   EmbeddingsFilter
)
```

Example:

```
embeddings_filter = EmbeddingsFilter(
```

```
    embeddings=embeddings,
    similarity_threshold=0.75
)
compression_retriever = ContextualCompressionRetriever(
    base_retriever=dense_retriever,
    base_compressor=embeddings_filter
)
```
It drops documents that are insufficiently related to the query according to embeddings.

11. Reranking

For cross-encoder reranking:

```
from langchain_classic.retrievers.document_compressors import (
    CrossEncoderReranker
)
from langchain_community.cross_encoders import (
    HuggingFaceCrossEncoder
)
```
Example:

```
model = HuggingFaceCrossEncoder
    model_name="cross-encoder/ms-marco-MiniLM-L6-v2"
)
reranker = CrossEncoderReranker(
    model=model,
    top_n=5
)
reranking_retriever = ContextualCompressionRetriever(
    base_retriever=dense_retriever,
    base_compressor=reranker
)
```
Current LangChain also exposes LLMListwiseRerank for LLM-based listwise reranking.

12. Sentence Window Retriever

This is the important one:

> **LangChain does not currently expose a canonical built-in class named SentenceWindowRetriever equivalent to LlamaIndex's SentenceWindowNodeParser + MetadataReplacementPostProcessor pattern.**

So in LangChain you implement it yourself.
Typical pieces:

```
from langchain_text_splitters import (
    RecursiveCharacterTextSplitter
)
```
or use a sentence tokenizer.
During ingestion, store:

```
metadata = {
    "sentence_id": 10,
    "window": "Sentence 8 ... Sentence 12"
}
```
Search the sentence embedding:

```
docs = vector_store.similarity_search(query, k=4)
```
Then replace the matched sentence with:

```
doc.metadata["window"]
```
So:

```
Search sentence
    ↓
Retrieve sentence
    ↓
Read window from metadata
    ↓
Return surrounding sentences
```
For **this exact technique**, LlamaIndex has the more direct built-in implementation; LangChain requires composition/custom code.

13. Multi-Hop Retrieval

There is no universal:

MultiHopRetriever(...)
class in core/current LangChain that represents the general multi-hop RAG concept.
For general multi-hop retrieval, use:

LangGraph
+
Retriever
+
LLM
+
state
Typical flow:

query
  ↓
retriever.invoke(query)
  ↓
LLM generates next query
  ↓
retriever.invoke(next_query)
  ↓
combine evidence
So for practical:

```
def first_hop(query):
   return retriever.invoke(query)
def generate_next_query(query, docs):
   ...

def second_hop(next_query):
   return retriever.invoke(next_query)
```

For a graph-backed corpus, LangChain does have:

GraphVectorStoreRetriever
and supports traversal:

```
retriever = graph_vectorstore.as_retriever(
   search_type="traversal",
   search_kwargs={
      "k": 6,
      "depth": 2
   }
)
```